

Count sort in C++

1. explain count sort algortihm in c++.2



1. explain count sort algortihm in c++

Sure — let's go through **Count Sort (Counting Sort)** step by step with a **clear explanation and example in C++** 👉

◆ What is Counting Sort?

Counting Sort is a **non-comparison-based** sorting algorithm that works well for sorting integers (or characters) when the **range of input values (k)** is not much larger than the number of elements (n).
It **counts** how many times each value appears, then uses that information to place elements in their correct positions.

◆ Steps of Counting Sort

1. **Find the maximum element** (let's call it `max`).
2. **Create a count array** of size `max + 1` initialized with zeros.
3. **Count occurrences** of each number in the input array.
4. **Compute prefix sums** (cumulative count) — so each index represents the position range of that element.
5. **Place each element** in the output array using the cumulative count.
6. **Copy output array** back to the original array.

◆ Example

Let's say:

```
css
Input: [4, 2, 2, 8, 3, 3, 1]
```

Step	Description	Array
1	Find max → 8	

Step	Description	Array
2	Make count[9] (0–8) all zeros	[0,0,0,0,0,0,0,0,0]
3	Count occurrences	[0,1,2,2,1,0,0,0,1]
4	Prefix sum	[0,1,3,5,6,6,6,6,7]
5	Build output array	[1,2,2,3,3,4,8]

◆ C++ Implementation

cpp

```
#include <iostream>
#include <vector>
using namespace std;

void countSort(vector<int>& arr) {
    int n = arr.size();
    if (n == 0) return;

    // 1. Find the maximum element
    int maxVal = *max_element(arr.begin(), arr.end());

    // 2. Create and initialize count array
    vector<int> count(maxVal + 1, 0);

    // 3. Count each element
    for (int num : arr)
        count[num]++;

    // 4. Compute prefix sum
    for (int i = 1; i <= maxVal; i++)
        count[i] += count[i - 1];

    // 5. Build output array
    vector<int> output(n);
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    // 6. Copy output back to original array
    arr = output;
}

int main() {
    vector<int> arr = {4, 2, 2, 8, 3, 3, 1};

    countSort(arr);

    cout << "Sorted array: ";
    for (int num : arr)
        cout << num << " ";
    cout << endl;
}
```

```
    return 0;
}
```

◆ Output

php

Sorted array: 1 2 2 3 3 4 8

◆ Time and Space Complexity

Type	Complexity
Time Complexity	$O(n + k) \rightarrow n = \text{number of elements, } k = \text{range of input}$
Space Complexity	$O(k)$ (for count array)
Stable	✓ Yes (if implemented using reverse traversal like above)
In-place	✗ No (requires extra array)

Would you like me to show a **version that supports negative numbers** too?